



KRESZ alapú tanulás segítő játék fejlesztése Unity keretrendszerben

Készítette

Fodor Győző Benedek

Programtervező Informatikus Bsc.

Témavezető

Troll Ede Mátyás

Tanársegéd

EGER, 2026

Tartalomjegyzék

Bevezetés	4
1. Technológiai áttekintés	5
1.1. Unreal Engine	5
1.2. Godot	6
1.3. Unity	6
1.3.1. Komponensek	6
1.4. Közlekedési szabályok	7
2. Rendszerterv	8
2.1. A rendszer célja	8
2.2. Telepítés	8
2.3. Fejlesztéshez használt eszközök	8
2.4. Funkcionális elvárások	8
3. Játékmenet	10
3.1. Szabad játék	10
3.2. Teszt játék	10
4. Saját szoftver	11
4.1. Fejlesztési idővonal	11
4.1.1. Tanulási időszak	11
4.1.2. Szoftver alapok	11
4.1.3. Unity alapok	11
4.1.4. Legfrissebb verzió	12
4.2. Menü	12
4.3. Játékprogram	12
4.3.1. Utak	13
4.3.2. Járművek	13
4.3.3. Kereszteződések	14

5. Fejlesztési lehetőségek	15
5.1. Textúrák és modellek	15
5.2. Új pályák és szituációk	15
5.3. Új szabályrendszerek	16
5.4. Szabálylista	16
5.5. Testre szabható feladatok	16
6. Tesztelés	17
6.1. Manuális	17
6.2. Egységteszt	17
Összegzés	18
Irodalomjegyzék	19
Ábrajegyzék	20

Bevezetés

A szakdolgozatom témáját akkor kellett választanom, amikor a szabadidőm nagy részét a jogosítványom megszerzésére fordítottam. A szabályok megértése és megtanulása egyszerű volt, de az úton, a közlekedési helyzetekben való alkalmazásuk már sokkal bonyolultabb. Az úgynevezett „3 pontos” feladatok, amik egy KRESZ vizsga jelentős részét alkotják, erre alapulnak. Az ilyen feladványok játékos gyakorlására akartam megoldást találni.

Egy olyan felületet szerettem volna létrehozni ami magába tudja foglalni egy közlekedési szituáció minden elemét. Fontos hogy a részletek könnyen felismerhetőek legyenek, és az egész jelenet interaktív legyen, segítve az adott helyzetben érvényesülő szabályok megértését. A tervem megvalósítására a Unity keretrendszert találtam a legalkalmasabbnak.

A szakdolgozat kódja, a hozzá tartozó build, és a játék működését bemutató videó a itt található:

Forráskód:

Build:

Videó:

1. fejezet

Technológiai áttekintés

A 20. század alkonyán a játékfejlesztők még sokkal kevesebben és kisebb erőforrásból dolgoztak, így sokkal gyakoribb volt, hogy a játékaikhoz saját motort fejlesztettek, mivel akkoriban nem sok kész, piaci játékmotor volt elérhető. Nagy segítség lehetett viszont, ha valaki meg tudott vásárolni egy kész kódbázist.

Ilyen volt például az *id Software* cég fejlesztette „id Tech” motor, melynek később több verziója is készült. Ezeket a motorokat használták az első meghatározó FPS (first person shooter) játékok mint például *DOOM*, *Quake*, *Wolfenstein* sorozatok alapjául.[1]

1.1. Unreal Engine

Az *Unreal Engine* az mai napig egyik legmeghatározóbb játékmotor. Az Epic Games játékfejlesztő cég 1998-ban készítette az *Unreal* nevű játékukhoz. Azóta 4-5 évente jelent meg új verzió, ami mindig jelentős változásokat hozott a aktuális játékiparba.

Eredetileg az *Unreal Engine* is az FPS genre kiszolgálására készült. Később viszont a TPP (third person perspective) féle kalandjátékok és rengeteg más 3D-s játék alapjául is szolgált, legyen az lövöldözős, kaland, nyílt világ, vagy történet műfajú.

A motor a mai napig C++ nyelven működik, 2012-ig Java-val társítva az *UnrealScript*-et használta, azonban ezt lecserélték és azóta a *Blueprint* vizuális programozási felület a fejlesztők barkácsolóasztala.

Az alapvetően ingyenes szoftver legújabb, 2022-es verziójának forráskódja a GitHub-on is elérhető, viszont a kereskedelmi felhasználás jogdíj köteles, amit az Epic Games a bevétel 5%-aként határozott meg. [4]

1.2. Godot

Egy másik szintén ingyenesen hozzáférhető és népszerű játékmotor a *Godot*. *Juan Linietsky* fejlesztette és adta ki először 2001-ben. A motor több néven is ismerté vált, majd 2014-ben felkerült GitHubra. Napjainkban a fejlesztés a 4.0-ás verzióánál tart, ahol a 3D-s felületet igyekeznek felzárkóztatni a versenytársakhoz.

A motor nem csak egy programozási nyelvhez kötött, de a három leggyakoribb a C++ és C#, valamint a GDScript. A GDScript a Godot saját rendszeréhez optimalizált objektum orientált programozási nyelv, a Pythonhoz hasonló struktúrával. A vizuális programozás opciója itt is megjelenik, régebben a *VisualScript*-ként, ez viszont a 4.0-ás verzióban már nem elérhető. [3]

1.3. Unity

A Unity a *Unity Technologies* fejlesztői által készített ingyenesen használható játékmotor. A motor támogatása kiterjed a számítógépeken kívül a konzolokra és okostelefonokra is. A 2D-s és 3D-s alkalmazások készítésére is alkalmas fejlesztői környezet hamar elterjedt a mobilfejlesztők és a feltörekvő, egy vagy több személyes kis fejlesztőcsapatok körében. A Unity felépítése könnyen megérthető és a C# programozási nyelv is gyorsan tanulható, emellett rengeteg oktatóvideó és kurzus is segíti a kezdőket az alapok elsajátításában.[4]

A fejlesztőcég először *Over The Edge Entertainment* jelent meg, csak 2007-ben nevezték át *Unity Technologies*-ra. [5]

A legjelentősebb fejlesztései:

- 2010: *Unity Asset Store*. Egy „Piac”, ahol a Unity-ben való fejlesztéshez használható fájlokat találhatsz ingyenesen és díjazva is.
- 2018: *Unity Hub*. Egy külön felület, ahol a Unity projekteket kezelheted és a motor különböző verziói közötti navigálást is megkönnyíti.

1.3.1. Komponensek

A Komponensek a Unity egyik fő résztvevői. A komponens tulajdonságai teszik lehetővé, hogy egy *GameObject* személyre szabható legyen. A *GameObject*-ek építik fel a játékokat, ezek a fő elemei a kész programnak. Sok komponens mint például a *Transform* vagy a *Collider*-ek, kevesebb funkciót ágyaznak magukba vagy rengetegszer felbukkanhatnak a program során. [6]

A Unity a saját komponensek létrehozását is lehetővé teszi, ezáltal kivételes szabadságot adva a fejlesztőnek. Amennyiben ezeknek a komponenseknek őse a *MonoBehaviour*

our osztály, az összes többi GameObject-re rácsatolhatóvá válnak és így új funkciókat adhatunk az adott objektumnak. [7]

1.4. Közlekedési szabályok

A Közúti Rendelkezők Egységes Szabályozása (rövidítve: KRESZ) az a közlekedési szabálygyűjtemény amely a Magyarország területén levő közutakon és közforgalom elől el nem zárt magánutakon folyó közlekedést szabályozza.

A közúti forgalom rendjét már a 19. században elkezdtek meghatározni. Az első magyar KRESZ egy 1910-ben született belügyminiszteri rendelet ami az azt egy évvel megelőző *Párizsi egyezmény* alapjaira készült. Ekkor még csak töredéke volt a mai szabályrendszernek. Ami viszont már ebben a korban és megjelent a KRESZ-ben az az autók műszaki vizsgálata és rendszámozása, a gépjárművezetők képzése, valamint az első négy közúti jelzőtábla:

- útkereszteződés
- kanyar
- vasúti átjáró
- bukkanó

1941-ben vezették be a jobboldali közúti közlekedést és ezzel együtt az úgynevezett „Jobbkéz szabályt”.

2. fejezet

Rendszerterv

2.1. A rendszer célja

A szakdolgozat célja egy olyan játék létrehozása, amiben a KRESZ vizsgák és tananyagokban szereplő „3-pontos” feladatokat játékos módon érthetőbbé, átláthatóbbá és tanulhatóbbá teszi. Fontos, hogy a játékos tudja tesztelni a tudását, a játék pedig interaktív és reszponzív legyen.

2.2. Telepítés

A nem szükséges telepíteni a játékszoftvert, egyből a kicsomagolás után indítható a mappában található alkalmazással.

2.3. Fejlesztéshez használt eszközök

- Unity: A játék készítéséhez használt fejlesztői környezet. Verzió: 2021.3.12f1
- Visual Studio: A programozói környezet, amely Unity támogatással rendelkezik. Verzió: Community 2022
- Visual Studio, WinForms: Vizuális fejlesztőkörnyezet a Visual Studio-n belül. A program kezdeti algoritmusainak a C# nyelvben való megvalósításra szolgált.

2.4. Funkcionális elvárások

- KRESZ alapú szabályrendszer: Egy olyan háttér algoritmus ami a szituációban szereplő objektumokat kezelni. Az algoritmus áthaladási sorrendet állít fel ami egyben a feladvány helyes megoldását is meghatározza.

- Interaktív játékos: A feladatot alkotó szereplők kattinthatóak legyenek és rendelkezzenek animált mozgással. A játék értesítse a felhasználót a hibás lépésről, vagy helyes megoldásról.
- Szabad játék: Végtelen ismétlődő randomizált feladványok ahol a játékos gyakorolhatja a közlekedési szituációkat.
- Nehézségi szintek: Különböző nehézségű feladványok a könnyebben követhető tanulási és gyakorlási görbéhez valamint a monotonitás elkerüléséhez.
- Teszt felület: Egy véges „Teszt” mód ahol a gyakorolt feladatokat adott megadott összesített sikerességi rátával kell megoldani a teszt teljesítéséhez. Támpont a tudás és gyakorlat fejlődésében.
- Pontrendszer: Mind a szabad játéknak, mind a teszt felületnek megfelelő pontrendszer ami a játékos eredményét rögzíti folyamatosan.

3. fejezet

Játékmenet

Az indítás után az első megálló a főmenü. Itt választhatjuk ki, hogy a „Play” gombra nyomva egy szabad játékba kezdünk, vagy a „Teszt”-el egy tesztet indítunk, ahol megmérettethetjük saját tudásunkat a gyakoroltak alapján.

3.1. Szabad játék

A szabad játék egy végtelen játékmód. Itt az összes elérhető variációból véletlenszerűen kiválasztott feladatokat kapunk amiket a közlekedési szituációban résztvevő autókra kattintva kell megoldanunk. Ha helyes sorrendben választjuk ki az autókat, a feladatot teljesítettük, és léphetünk tovább a következőre.

Minden sikeresen megoldott feladat után egy pont jár, az ide tartozó pontrendszer a játékos aktuális játéka alatt helyesen megoldott feladványait tartja számon egész addig még a játékot be nem fejezik.

3.2. Teszt játék

A „Teszt” gombra kattintva egy teszt játék veszi kezdetét. A *KRESZ vizsga* mintájára, adott darab feladványt kell megoldania a játékosnak. Egy teszt feladatai már meghatározottabbak mint a szabad játék szituációi. Van több nehézségi szint. A különböző nehézségű feladatokat arányosan osztja el a rendszer, így a könnyebb és nehezebb szituációkkal egyaránt meg kell tudni birkózni a teszt teljesítéséhez.

Itt is van pontrendszer, de itt már nem csak a rekordok fenntartása végett. A teszt sikeres teljesítéséhez, az összesen megszerezhető maximális pontszám legalább 90%-át kell elérnie a játékosnak, de persze mindig lehet törekedni a tökéletes eredményre.

4. fejezet

Saját szoftver

4.1. Fejlesztési idővonal

A programom teljes kifejlesztése összesen 2 évet vett igénybe. Ez idő alatt voltak aktívabb és passzívabb időszakok is, volt amikor egy-egy áttörés után rövid időre megnövekedett az elért mérföldkövek száma. A program több alakot is öltött.

4.1.1. Tanulási időszak

Az első félév alatt a program elég csekély formában volt jelen. Ebben az időszakban a fókusz a projekthez használt programozási nyelv megértésén és elsajátításán volt.

Elkészültek a program első tervei amik később támpontot adtak a fejlesztés során. Már itt előre sok elméleti problémára próbáltam megoldást keresni, ötletelni, hogy ha esetleg ezek a problémák később felmerülnének, ne érjen akkora meglepetés.

4.1.2. Szoftver alapok

A program első verziója még teljesen a *Visual Studio*-ban, egy „WinForms” felületre készült. Itt jött létre a program alapjául szolgáló algoritmus első, primitív verziója, ami a szituációk előre megadott adatokból volt képes kiszámolni az áthaladás megfelelő sorrendjét.

Ebben az állapotban még csak az alapszabályoknak megfelelően működött, nem tudta még kezelni a nem teljesen lineáris megoldásokat. A kiválasztott paraméterek alapján felállított egy sorrendet és azt adta vissza a kezelőfelületen, nem volt még vizuális megjelenítés, csak számok és szavak.

4.1.3. Unity alapok

A második félév végén nagy áttörés történt. A program átköltözött a Unity keretrendszerébe. Az eddig kifejlesztett algoritmus megkapta az interaktív felületet az első

vizuális megjelenítését. Ez már a GitHubra is felkerült.

A harmadik félévben rájöttem, hogy habár a programíráshoz már megvan a tudásom, a Unity sajátos eszközkészletét még messze nem ismerem eléggé. Tanuló anyagokat kerestem a témában, gyakorló projekteket hoztam létre, és teljesítettem egy Unity specifikus kurzust is, hogy a megfelelő szintre hozzam a tudásomat, mindemellett folyamatosan integráltam az ezalatt tanult módszereket és újdonságokat a programba.

4.1.4. Legfrissebb verzió

Az utolsó pár hónapban is sok módosításon esett átt a játék. Elkészült egy kezdetleges „feedback” rendszer ami a későbbiekben lényegesen megkönnyítette a hibakeresést és a javítást, hiszen a konzolon folyamatosan megjelentek a „real time” adatai az aktív feladatnak. Később pedig az eredetileg *placeholder*-nek szánt modellek is lecserélésre kerültek.

Az algoritmus finomodott és immár képessé vált a nem lineáris sorrendű megoldások kezelésére. Ez gyakorlatban annyit jelent, hogy az algoritmus folyamatosan újravizsgálja a szituációt minden lépés után, és megállapítja a következő helyes válaszlehetőségeket. Erről még bővebben később.

4.2. Menü

A játékban kétféle menü van. A Főmenü és a köztes játékmenü.

- Főmenü, egyszerű 3 gombbal rendelkezik:
 - Play: Elintíti a szabad játék módot.
 - Test: Elindítja a teszt játék módot.
 - Exit: Kilép a játékból, bezárja azt.
- Játékmenü
 - Next Scene: Tovább lép a következő feladatra.
 - Back to Menu: Kilép a főmenübe. Ez mindkét játékmód esetében az alap helyzetbe helyezi az előrehaladást.

4.3. Játékprogram

A projekt fő része, játék alapjaként szolgáló program, 3 fő részből áll:

- Utak

- Járművek
- Kereszteződések

Az utak és a járművek alkotják a kereszteződések, utóbbiak pedig az előbbiek adataiból elvégzik a szituációk létrehozásához kellő számításokat, és kezelik a megoldó algoritmust.

4.3.1. Utak

Elindítjuk a játékot, a jelenet futni kezd, a *Road* osztály az első megálló.

- Létrehozza a *Node*-okat.
 - *StartNode*: Az út objektumok jobb oldalán található Node. Ez a járművek kiindulási pontja, itt jelennek meg először.
 - *EndNode*: Az út bal oldalán található Node. Ide érkeznek a járművek, ez a kilépési pontjuk. Ez a Node határozza meg az adott jármű irányát.
- Megjelöli a felhasznált út objektumok irányát. Innen érkezik az az információ, hogy a jármű melyik utak Start- és EndNodejai közül választhat.
- Kiosztásra kerülnek az út rangok. Később ez alapján érvényesülnek az elsőbbség, és jobbkéz szabályok. Ha egy út magasabb rangú mint a szomszédja akkor az azon úton haladó jármű elsőbbséget élvez az alacsonyabb rangú úton állóval szemben. Ha az utak egyenrangúak, a jobbkéz szabály érvényesül.
- A *PlaceVehicle* metódus a *Vehicle* osztályból beérkező adatokkal egy járművet hoz létre, amit elhelyez az út *StartNode*-ján.

4.3.2. Járművek

A második megálló a *Vehicle* osztály.

- Létrejön egy jármű objektum, és a következő paramétereket kapja meg:
 - *Kiindulópont*: Egy olyan *StartNode* ami szabad.
 - *Irány*: Lehet előre, balra vagy jobbra.
 - *Cél*: Az alapján, hogy melyik irányba szeretne haladni, megkapja az annak megfelelő *EndNode* adatait. EndNode lehet közös, StartNode nem.
 - Jobb szomszéd: Megvizsgálja van e tőle jobbra másik jármű, hiszen egyenrangú kereszteződés esetén el kell engednie.

- Rang: Ez a járművek rangja, a játék közben folyton változik ahogy a járművek megközelítik, majd elhagyják a kereszteződést. Ez adja meg az áthaladási sorrendet. Azért kell folyamatosan frissülnie, mert némelyik szituációban lehet több aktuálisan helyes lépés is, valamit egy lépés után lehet olyan jármű aki elveszíti a szomszédját.
- Az animációkhoz használt komponensek is itt csatolódnak az objektumokhoz. [7]

4.3.3. Kereszteződések

A kereszteződéseknek két része van. Az *IntersectionBuilder* osztály és az *IntersectionSolver* osztály.

- *IntersectionBuilder*: „Megépíti” a kereszteződések. Felhasználva a kész út és jármű objektumokat és azok paramétereit, az elemeket összerakva vizualizálja a kész *Intersection* objektumot. Egy ilyen objektumnak legalább két járműből és három útszakaszból kell állnia, hogy megfelelő kereszteződést alkosson.
- *IntersectionSolver*:
 - Kezeli a megoldó algoritmust, a folyamatosan frissülő áthaladási sorrendet és az alapján Log-ol a konzolra is.
 - Kezeli a rövid és hosszútávú eredményeket a pontszámrendszerrel a szabad játék és a teszt játék esetében is.

5. fejezet

Fejlesztési lehetőségek

Rengeteg potenciál van még a programban. Sok olyan ötletem volt és van ami még nem valósult meg, de elméletben már létezik.

5.1. Textúrák és modellek

A projekt alatt sokkal több figyelmet fordítottam a program működési folyamatainak megbízhatóságára mint a megjelenésére, így van még pár olyan része a játéknak amire ráférne egy ráncfelvarrás:

- Autók: A játék kis szereplői, a közlekedésben résztvevő járművek habár aranyosak a maguk alacsony bonyolultságú kinézetükkel, sokat hozzátenne a játékélményhez az alaposabban kidolgozott autó modellek alkalmazása. A fejlesztendő textúrák és modellek közül ez számomra a legfontosabb, és ez egy hozzáértő új csapat tag bevonásával könnyen megvalósítható lenne.
- Gombok és feliratok: A játék legtöbb felirata és gombjai egyszerű kinézettel rendelkeznek. Ezeknek a cseréje szintén nagyon feldobná a játékot és jobb esztétikai élményt adna.
- Környezet: A játékkörnyezet legalább annyira fontos mint a résztvevő autóké. A környezet kidolgozottsága egyeznie kell a járművekével, hogy az autók ne tűnjenek ki a helyzetből jobban mint kellene.

5.2. Új pályák és szituációk

Szeretném, hogy a játék ne legyen túlságosan egyoldalú. Ennek az eléréséhez a jövőben új pályákat tervezek hozzáadni amik egy fokkal feljebb emelik az alap kereszteződés alapú pályákat.

A kereszteződések mellett a következő lépés a kétirányú úttestek, körforgalmak és lámpás kereszteződések, esetleg további résztvevő járművek integrálása lesz.

A nehézségi szintet tovább emelve, a járműveknek *Collidert* adva az áthaladási animációkkal egyszerű baleseteket is szimulálhatnánk, ami türelemre és odafigyelésre is tanítaná a játékost.

5.3. Új szabályrendszerek

Az új kétirányú, körforgalom és lámpás kereszteződések beépítésével új szabályrendszerekre is szükség lesz.

- Kétirányú úttest: Az ilyen típusú pályához kell egy olyan algoritmus ami tudja szimulálni az előzés, kikerülés, úttest elhagyása és a megkülönböztető jelzéssel haladó jármű érkezését, és az ezek által létrehozott szimulációban a kereszteződésekhez hasonlóan áthaladási sorrendet és megoldást generálni.
- Körforgalom: A Körforgalom egy egyszerű de annál furfangosabb és fontos része a közúti közlekedésnek. A beépítéséhez egy külön elsőbbség adási rendszer, és az irányjelzés és az azt figyelő algoritmus is része kell, hogy legyen.
- Lámpás kereszteződés: A lámpás kereszteződés a mostanihoz hasonló de egy lépcsőfokkal ellenőrzöttebb rendszer, viszont fontosnak érzem, hogy a későbbiekben ez is a része legyen a programnak.
- Új járművek: A KRESZ-ben gyakran előfordulnak speciális járművek és résztvevők amikre a forgalom különös figyelemmel vagy akár konkrét szabályokkal reagál. Ilyen például:
 - Vonat: Vasúti átkelőhely.
 - Villamos: Kereszteződés villamossínnel, opcionális érkező villamossal.
 - Megkülönböztető jelzés: Rendőr, mentő, tűzoltó! A kereszteződésekbe és többi pályafarmába is opció.
 - Gyalogos: Sok esetben nem teljesen egyértelmű, hogy egy gyalogosnak mikor van elsőbbsége az autókkal szemben, és ez könnyen bezavarhat egy-egy szituációban, így extra nehezítésként a gyalogosok jelenlétét is lehetne adni a feladathoz.

5.4. Szabálylista

A játékon belül egy olyan felület, ahol a játék során fellelhető közlekedési szabályokat magyarázattal és illusztrációval vannak feltüntetve. Így ezeket a szabályokat a prog-

ramon belül tanulmányozhatja a felhasználó, ha valamelyiknek szeretne utánanézni esetleg egy elrontott feladat után.

5.5. Testreszabható feladatok

Még az első bővítési ötleteim között volt a szituációk testreszabhatósága. Ez azt jelenté, hogy egy külön játékmódban a felhasználó beállíthatja, hogy:

- Milyen kereszteződés vagy közlekedési helyzet legyen a szituáció alapja.
- Milyen és hány résztvevője legyen az adott feladatnak.
- Az adott résztvevők milyen irányban haladjanak tovább, rendelkezzenek e bármilyen plusz joggal.

A beállítások véglegesítése után a játékos megoldhatja a saját szituációját aztán pedig új feladat létrehozásába kezdhet.

6. fejezet

Tesztelés

szöveg

6.1. Manuális

szöveg

6.2. Egységteszt

szöveg

Összegzés

Véleményem szerint ebből a programból nagyon hasznos eszköz válhat azok számára akik vezetni szeretnének tanulni és kell egy kis segítség a szabályok megértésében.

Irodalomjegyzék

- [1] WIKIPÉDIA: *Id Software*, https://en.wikipedia.org/wiki/Id_Software, Megtekintve: 2026.04.05.
- [2] WIKIPÉDIA: *Unreal Engine*, https://en.wikipedia.org/wiki/Unreal_Engine, Megtekintve: 2026.04.05.
- [3] WIKIPÉDIA: *Godot (game engine)*, [https://en.wikipedia.org/wiki/Godot_\(game_engine\)](https://en.wikipedia.org/wiki/Godot_(game_engine)), Megtekintve: 2026.04.05.
- [4] WIKIPÉDIA: *Unity (game engine)*, [https://en.wikipedia.org/wiki/Unity_\(game_engine\)](https://en.wikipedia.org/wiki/Unity_(game_engine)), Megtekintve: 2026.04.05.
- [5] WIKIPÉDIA: *Unity Technologies*: https://en.wikipedia.org/wiki/Unity_Technologies, Megtekintve: 2026.04.06.
- [6] UNITY DOCUMENTATION: *Introduction to components*: <https://docs.unity3d.com/Manual/Components.html>, Megtekintve: 2026.04.08.
- [7] UNITY DOCUMENTATION: *MonoBehaviour*: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>, Megtekintve: 2026.04.08.
- [8] WIKIPÉDIA: *KRESZ*: <https://hu.wikipedia.org/wiki/KRESZ>, Megtekintve: 2026.04.18.

Ábrajegyzék

Nyilatkozat

Alulírott, büntetőjogi felelősségem tudatában kijelentem, hogy az általam benyújtott, című szakdolgozat önálló szellemi termékem. Amennyiben mások munkáját felhasználtam, azokra megfelelően hivatkozom, beleértve a nyomtatott és az internetes forrásokat is.

Aláírással igazolom, hogy az elektronikusan feltöltött és a papíralapú szakdolgozatom formai és tartalmi szempontból mindenben megegyezik.

Eger, 2026. március 14.

aláírás

A szakdolgozat megírása után ezt a nyilatkozatot kell a végéhez csatolni a következő módon:

1. A `nyilatkozat` mappában a `nyilatkozat.tex` fájlt töltsse ki és fordítsa le pdf-be!
2. A `nyilatkozat.pdf` fájlt nyomtassa ki, majd írja alá!
3. Szkennelje be pdf formátumba, majd ezzel írja felül a `nyilatkozat` mappában a `nyilatkozat.pdf` fájlt!
4. A szakdolgozat forrásfájljában legyen betöltve a `pdfpages` csomag. Az utolsó két sor legyen az alábbi:

```
\includepdf{nyilatkozat/nyilatkozat.pdf}  
\end{document}
```

5. Ezután fordítsa le a szakdolgozatot pdf-be!